

**Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro**

Nombre del trabajo:

Manual Técnico – Sistema de Detección de Fuego

Asignatura:

Inteligencia Artificial y Diseño Electrónico Basado en Sistemas
Embebidos

Profesor:

Dr. Ángel Mario Lerma Sánchez,
Dr. Daniel López Piña.

Integrantes:

Hernández Ruiz Haydee Michelle,
Montoya Garza Luis Ángel,
Silva Sánchez Yamilka Arely,
Rueda Martínez Alison Michelle.

Grado: 8° Grupo: E

Ciudad Mante, Tamaulipas

Mayo 2026

Índice

1. Introducción	4
2. Requisitos del Sistema	4
2.1 Hardware necesario	4
2.2 Software base	4
2.3 Librerías de Python	5
3. Instalación del Sistema	5
3.1 Preparar el entorno	5
3.2 Instalar librerías de Python	5
3.3 Colocar los archivos del proyecto	6
3.4 Configurar el token del bot de Telegram	6
4. Configuración de Servicios systemd	6
4.1 fireguard-detect.service	6
4.2 fireguard-setup.service	7
4.3 aegnis-led.service	7
4.4 Instalar y activar los servicios	8
5. Arquitectura del Sistema	10
5.1 Diagrama de componentes	10
5.2 Flujo de datos	10
6. Descripción de Archivos y Funciones	10
6.1 detector.py	10
6.2 app.py	11
6.3 led.py	12
6.4 config.json	13
7. Modelo de Detección — Proceso de Entrenamiento	13

7.1 Dataset.....	13
7.2 Entrenamiento con YOLOv8	13
7.3 Exportar a ONNX	14
7.4 Métricas obtenidas	14
8. Configuración de Red WiFi.....	15
9. Conexión del LED RGB	15
10. Diagnóstico y Solución de Problemas	16
12. Resultados del Modelo	16
12.1 Métricas finales en el conjunto de prueba	16
12.2 Curvas de entrenamiento	17
12.3 Curva Precisión-Confianza	17
12.4 Curva Precisión-Recall (Área bajo la curva)	18
12.5 Curva Recall-Confianza	18
12.6 Matriz de confusión	19
12.7 Ejemplos de detección	20
13. Referencias	22

1. Introducción

Este manual técnico documenta el proceso completo de replicación del sistema Aegnis, desde la preparación del hardware hasta la puesta en marcha de todos los servicios. Está dirigido a personas con conocimientos en sistemas Linux, Python y Raspberry Pi que deseen reproducir el proyecto o realizar modificaciones sobre él.

El sistema está compuesto por tres módulos de software independientes que se comunican mediante archivos compartidos en el sistema de archivos, un bot de Telegram como interfaz de control remoto, y un servidor web Flask para el monitoreo visual en red local.

2. Requisitos del Sistema

2.1 Hardware necesario

Componente	Especificación mínima	Observaciones
Raspberry Pi	Raspberry Pi 4B+ o superior	Se usó con Debian Trixie (Raspberry Pi OS)
Cámara	Cualquier cámara USB	Se usó Generalplus GENERAL WEBCAM
LED RGB	LED RGB de ánodo común	Pines GPIO 17 (R), 27 (G), 22 (B)
MicroSD	16 GB clase 10 o superior	Para sistema operativo y archivos
Fuente de poder	5V	Fuente oficial Raspberry Pi recomendada
Conexión de red	WiFi 802.11n o superior	Para Telegram y acceso SSH

2.2 Software base

- Sistema operativo: Raspberry Pi OS basado en Debian Trixie (64-bit recomendado)
- Python 3.11 o superior
- NetworkManager (nmcli) para gestión de red
- systemd para gestión de servicios
- Git para clonar repositorios de drivers si es necesario

2.3 Librerías de Python

Librería	Versión usada	Uso en el sistema
ultralytics	8.x	Carga y ejecución del modelo YOLOv8
opencv-python	4.x	Captura de cámara y procesamiento de frames
flask	3.x	Servidor web local y stream MJPEG
requests	2.x	Comunicación con la API de Telegram
RPi.GPIO	0.7.x	Control del LED RGB por GPIO
onnxruntime	1.25.x	Inferencia del modelo ONNX en CPU

3. Instalación del Sistema

3.1 Preparar el entorno

Actualizar el sistema operativo antes de instalar cualquier dependencia:

```
sudo apt update && sudo apt upgrade -y
```

Instalar dependencias del sistema:

```
sudo apt install -y python3-pip python3-venv git libopencv-dev
```

Crear el directorio del proyecto y un entorno virtual:

```
mkdir -p /home/fisical/sistema_incendios
cd /home/fisical/sistema_incendios
python3 -m venv venv
source venv/bin/activate
```

3.2 Instalar librerías de Python

```
pip install ultralytics opencv-python flask requests RPi.GPIO onnxruntime
```

Nota: En Raspberry Pi la instalación de ultralytics y onnxruntime puede tardar varios minutos. Si aparece error de memoria durante la compilación, añadir '--no-cache-dir' al comando pip.

3.3 Colocar los archivos del proyecto

Copiar los siguientes archivos al directorio /home/fisica1/sistema_incendios/:

Archivo	Descripción
app.py	Servidor Flask + bot de Telegram
detector.py	Detector de incendios con YOLO
led.py	Controlador del LED RGB
best.onnx	Modelo entrenado en formato ONNX
rpi-setup.html	Página web de configuración inicial

Crear el archivo de configuración inicial:

```
echo '{"wifi_ssid":"","chat_id":"","owner_vinculado":false,"hijos":[]}' > config.json
```

3.4 Configurar el token del bot de Telegram

El token del bot está definido como constante en app.py y detector.py. Para usar un bot propio, reemplazar el valor de TOKEN_BOT en app.py y TOKEN en detector.py con el token obtenido desde @BotFather en Telegram.

4. Configuración de Servicios systemd

El sistema utiliza tres servicios de systemd para garantizar que cada módulo arranque automáticamente al encender la Raspberry Pi y se reinicie en caso de fallo.

4.1 fireguard-detect.service

Servicio que ejecuta el detector de incendios. Se configura para arrancar después de que la red esté disponible y después de que fireguard-setup esté corriendo.

```
[Unit]
Description=FireGuard - Detector de incendios (cámara + YOLO)
After=network.target fireguard-setup.service
Wants=network-online.target

[Service]
Type=simple
User=fisical
WorkingDirectory=/home/fisical/sistema_incendios
Environment=DISPLAY=
ExecStart=/usr/bin/python3 /home/fisical/sistema_incendios/detector.py
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

4.2 fireguard-setup.service

Servicio que ejecuta el servidor Flask y el bot de Telegram. Corre como root para poder usar el puerto 80 sin configuración adicional.

```
[Unit]
Description=FireGuard - Servidor de configuración (Flask)
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/home/fisical/sistema_incendios
ExecStart=/usr/bin/python3 /home/fisical/sistema_incendios/app.py
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

4.3 aegnis-led.service

Servicio que controla el LED RGB. Corre como root para acceder a los pines GPIO.

```
[Unit]
Description=Aegnis - Controlador LED RGB
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/home/fisical/sistema_incendios
ExecStart=/usr/bin/python3 /home/fisical/sistema_incendios/led.py
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal
```

```
[Install]
```

```
WantedBy=multi-user.target
```

4.4 Instalar y activar los servicios

Copiar los tres archivos .service al directorio de systemd y habilitarlos:

```
sudo cp fireguard-detect.service /etc/systemd/system/  
sudo cp fireguard-setup.service /etc/systemd/system/  
sudo cp aegnis-led.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable fireguard-detect fireguard-setup aegnis-led  
sudo systemctl start fireguard-detect fireguard-setup aegnis-led
```

```
fisical@fisical:~$ systemctl list-units --type=service --state=running | grep -E "fireguard|aegnis"  
aegnis-led.service      loaded active running Aegnis - Controlador de LED RGB  
fireguard-detect.service loaded active running FireGuard - Detector de incendios (cámara + YOLO)  
fireguard-setup.service loaded active running FireGuard - Servidor de configuración (Flask)  
fisical@fisical:~$
```


Verificar que los servicios estén corriendo:

```
sudo systemctl status fireguard-detect
sudo systemctl status fireguard-setup
sudo systemctl status aegnis-led
```



```
2.168.1.136 (WayVNC) - RealVNC Viewer
fisical@fisical: ~
File Edit Tabs Help
fisical@fisical:~$ sudo systemctl status fireguard-setup fireguard-detect aegnis-led --no-pager
• fireguard-setup.service - FireGuard - Servidor de configuración (Flask)
  Loaded: loaded (/etc/systemd/system/fireguard-setup.service; enabled; preset: enabled)
  Active: active (running) since Fri 2026-05-08 18:55:56 CST; 3h 27min ago
  Invocation: 4d7537ab1f3643009c3bdf5b65d89087
  Main PID: 932 (python3)
  Tasks: 2 (limit: 3906)
  CPU: 1.037s
  CGroup: /system.slice/fireguard-setup.service
          └─932 /usr/bin/python3 /home/fisical/sistema_incendios/app.py

May 08 18:55:56 fisical systemd[1]: Started fireguard-setup.service - Fire_ask).
May 08 18:55:58 fisical python3[932]: @ Servidor iniciado en http://0.0.0.0:80
May 08 18:55:58 fisical python3[932]: * Serving Flask app 'app'
May 08 18:55:58 fisical python3[932]: * Debug mode: off
May 08 18:55:58 fisical python3[932]: WARNING: This is a development server..ead.
May 08 18:55:58 fisical python3[932]: * Running on all addresses (0.0.0.0)
May 08 18:55:58 fisical python3[932]: * Running on http://127.0.0.1:80
May 08 18:55:58 fisical python3[932]: * Running on http://127.0.0.1:80
May 08 18:55:58 fisical python3[932]: Press CTRL+C to quit

• fireguard-detect.service - FireGuard - Detector de incendios (cámara + YOLO)
  Loaded: loaded (/etc/systemd/system/fireguard-detect.service; enabled; preset: enabled)
  Active: active (running) since Fri 2026-05-08 18:55:56 CST; 3h 27min ago
  Invocation: b9878009a9f64ed7bd051545f05c07ad
  Main PID: 933 (python3)
  Tasks: 15 (limit: 3906)
  CPU: 3min 23.184s
  CGroup: /system.slice/fireguard-detect.service
          └─933 /home/fisical/sistema_incendios/venv/bin/python3 /home/fisic..

May 08 18:55:56 fisical systemd[1]: Started fireguard-detect.service - Fir_OL0).
May 08 22:22:45 fisical python3[933]: @ FireGuard detector iniciando...
May 08 22:22:45 fisical python3[933]: ✓ Modelo cargado correctamente.
May 08 22:22:45 fisical python3[933]: ✓ Cámara inicializada.
May 08 22:22:45 fisical python3[933]: @ FireGuard corriendo en modo headl_deo).
May 08 22:22:45 fisical python3[933]: Loading /home/fisical/sistema_incendi.e...
May 08 22:22:45 fisical python3[933]: 2026-05-08 22:22:45.642727765 [W:onnx_dor"
May 08 22:22:45 fisical python3[933]: 2026-05-08 22:22:45.645634857 [W:onnx_dor"
May 08 22:22:45 fisical python3[933]: Using ONNX Runtime 1.25.1 with CPUExe_ider

• aegnis-led.service - Aegnis - Controlador de LED RGB
  Loaded: loaded (/etc/systemd/system/aegnis-led.service; enabled; preset: enabled)
  Active: active (running) since Fri 2026-05-08 18:55:56 CST; 3h 27min ago
  Invocation: 0a373b3d51f84aeeba5b3d0c2c760275
  Main PID: 934 (python3)
  Tasks: 1 (limit: 3906)
  CPU: 257ms
  CGroup: /system.slice/aegnis-led.service
          └─934 /home/fisical/sistema_incendios/venv/bin/python3 /home/fisic..

May 08 18:55:56 fisical systemd[1]: Started aegnis-led.service - Aegnis - C..RGB.
Hint: Some lines were ellipsized, use -l to show in full.
fisical@fisical:~$
```

Ver los logs en tiempo real de cualquier servicio:

```
journalctl -u fireguard-detect -f
```

5. Arquitectura del Sistema

5.1 Diagrama de componentes

El sistema está compuesto por tres procesos independientes que se comunican mediante archivos compartidos en el sistema de archivos temporal /tmp/:

Archivo compartido	Escrito por	Leído por
/tmp/aegnis_frame.jpg	detector.py (frame anotado)	app.py (stream MJPEG)
/tmp/aegnis_camara_ok	detector.py (flag de cámara activa)	led.py (verificar estado de cámara)
config.json	app.py (bot y configuración)	detector.py (chat_ids para alertas)

5.2 Flujo de datos

- La cámara USB es capturada por detector.py mediante cv2.VideoCapture(0) a 20 fps.
- El resultado se clasifica por nivel de riesgo y se dibuja sobre el frame, que se escribe en /tmp/aegnis_frame.jpg.
- app.py lee continuamente ese archivo y lo sirve como stream MJPEG en la ruta /video.
- Cuando hay detecciones activas y ha pasado el cooldown de 30 segundos, se guarda alerta.jpg y se envía a todos los chat_ids registrados en config.json.
- led.py consulta cada 5 segundos si /tmp/aegnis_camara_ok fue modificado en los últimos 30 segundos y si 8.8.8 responde ping, actualizando el LED según el estado combinado.

6. Descripción de Archivos y Funciones

6.1 detector.py

Módulo principal de detección. Corre de forma independiente como servicio y no importa ni depende de app.py.

Función / Variable	Descripción
TOKEN	Token del bot de Telegram para envío de alertas
MODELO	Ruta al archivo best.onnx del modelo entrenado

CONFIANZA_MINIMA	Umbral mínimo de confianza para aceptar detecciones (0.5)
COOLDOWN_ALERTAS	Segundos mínimos entre alertas consecutivas (30)
cargar_chat_ids()	Lee config.json y retorna lista de todos los chat_ids registrados
enviar_mensaje()	Envía texto a un chat_id específico vía API de Telegram
enviar_foto_a_todos()	Envía imagen con caption a todos los usuarios vinculados
abrir_camara()	Inicializa cv2.VideoCapture(0) y retorna el objeto cap
procesar_frame()	Ejecuta inferencia YOLO, clasifica detecciones por nivel, dibuja anotaciones y guarda /tmp/aegnis_frame.jpg
enviar_alerta_incendio()	Determina el nivel máximo de riesgo y envía foto con caption a todos los usuarios
Loop principal	Captura frames, lanza hilos de procesamiento cada 3 frames y gestiona reconexión de cámara

6.2 app.py

Servidor Flask y bot de Telegram. Corre dos componentes en paralelo: el servidor web en el hilo principal y el bot en un hilo daemon independiente.

Función / Ruta	Descripción
cargar_config()	Lee config.json y retorna el diccionario de configuración
guardar_config()	Serializa y escribe el diccionario de configuración en config.json
generar_codigo()	Genera código alfanumérico aleatorio de 6 caracteres para vinculación
enviar_telegram()	Envía mensaje de texto a un chat_id vía API de Telegram
ip_local()	Obtiene la IP local actual de la Raspberry Pi mediante hostname -I
wifi_actual()	Consulta nmcli para obtener el SSID de la red actualmente conectada
escanear_redes()	Ejecuta nmcli para escanear redes WiFi disponibles y retorna lista de SSIDs

GET /	Sirve la página rpi-setup.html de configuración inicial
GET /video	Stream MJPEG que lee continuamente /tmp/aegnis_frame.jpg
GET /monitor	Página HTML con el stream de video embebido e información del sistema
GET /api/status	JSON con estado completo del sistema
GET /api/wifi/scan	JSON con lista de redes WiFi disponibles
POST /api/wifi/connect	Conecta a una red WiFi recibida como JSON {ssid, pass}
GET /api/telegram/code	Genera y almacena código de vinculación para el padre
bot_loop()	Hilo daemon que consulta getUpdates de Telegram y procesa comandos

6.3 led.py

Controlador del LED RGB. Corre de forma completamente independiente consultando el estado del sistema mediante ping y el archivo flag de la cámara.

Función / Variable	Descripción
PIN_R, PIN_G, PIN_B	Números de pines GPIO BCM para cada canal del LED (17, 27, 22)
INTERVALO	Segundos entre cada ciclo de verificación de estado (5)
set_led(r, g, b)	Activa/desactiva cada canal del LED. En modo simulación imprime el estado
iniciar_parpadeo()	Lanza hilo daemon que alterna el LED entre encendido y apagado
detener_parpadeo()	Detiene el hilo de parpadeo y apaga el LED
hay_internet()	Ejecuta ping a 8.8.8.8 y retorna True si el código de retorno es 0
hay_camara()	Verifica que /tmp/aegnis_camara_ok existe y fue modificado en los últimos 30s
fallos_internet	Contador de fallos consecutivos de ping (cambia a rojo solo tras 3 fallos)

loop()	Bucle principal que evalúa estado, cambia el LED si hay diferencia respecto al estado anterior
--------	--

6.4 config.json

Archivo de configuración persistente del sistema. Se genera automáticamente al primer arranque si no existe. Estructura:

```
{
  "wifi_ssid": "NOMBRE_RED",
  "chat_id": "123456789",
  "owner_vinculado": true,
  "hijos": [
    {"chat_id": "987654321", "nombre": "Usuario"}
  ]
}
```

7. Modelo de Detección — Proceso de Entrenamiento

7.1 Dataset

Se utilizó el dataset Indoor Fire Smoke Dataset disponible públicamente en Zenodo:

- URL: <https://zenodo.org/records/15826133>
- DOI: <https://doi.org/10.5281/zenodo.15826133>
- Autor: Putra, A. K. (2025)
- Total: 5,000 imágenes divididas en train (3,500), valid (750) y test (750)
- Clases: 0 = fuego, 1 = humo

7.2 Entrenamiento con YOLOv8

El entrenamiento se realizó con la librería Ultralytics YOLOv8. Instalar y entrenar:

```
from ultralytics import YOLO
```

```
# Cargar modelo base YOLOv8 nano (más ligero, ideal para Raspberry Pi)
model = YOLO('yolov8n.pt')
```

```
# Entrenar
```

```
results = model.train(
    data='/content/drive/MyDrive/Indoor Fire Smoke/data.yaml',
    epochs=70,
    imgsz=640,
    batch=16,
    patience=10,          # detiene si no mejora
    save=True,
```

```

    project='/content/drive/MyDrive/Indoor Fire Smoke',
    name='v3',
    exist_ok=True
)

from ultralytics import YOLO

# Cargar modelo base YOLOv8 nano (más ligero, ideal para Raspberry Pi)
model = YOLO('yolov8n.pt')

# Entrenar
results = model.train(
    data='/content/drive/MyDrive/Indoor Fire Smoke/data.yaml',
    epochs=70,
    imgsz=640,
    batch=16,
    patience=10,          # detiene si no mejora en 10 epochs (early stopping)
    save=True,
    project='/content/drive/MyDrive/Indoor Fire Smoke',
    name='v3',
    exist_ok=True
)

print('\n✅ Entrenamiento completado')
print('Mejor modelo guardado en: /content/drive/MyDrive/Indoor Fire Smoke/v3/weights/best.pt')

```

Figura. Parte de modelado en Google Colab

El archivo dataset.yaml debe tener la siguiente estructura:

```

path: /ruta/al/dataset
train: images/train
val: images/valid
test: images/test
nc: 2
names: ['fire', 'smoke']

```

7.3 Exportar a ONNX

Una vez entrenado, exportar el modelo al formato ONNX para inferencia en CPU:

```

model = YOLO('runs/detect/aegnis_fire/weights/best.pt')
model.export(format='onnx', opset=12)

```

El archivo best.onnx generado se copia al directorio del sistema en la Raspberry Pi:

```

scp best.onnx fisical@<IP_RPI>:/home/fisical/sistema_incendios/best.onnx

```

7.4 Métricas obtenidas

Métrica	Valor
mAP50	0.873
mAP50-95	0.548
Precision	0.883
Recall	0.788
AP fuego (clase 0)	0.874
AP humo (clase 1)	0.862

8. Configuración de Red WiFi

La Raspberry Pi gestiona la conectividad WiFi mediante NetworkManager. Para agregar una red manualmente desde la terminal:

```
nmcli connection add type wifi ssid 'NOMBRE_RED' \
  wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk 'CONTRASENA' \
  connection.id 'nombre-conexion' \
  ifname wlan0
nmcli connection up 'nombre-conexion'
```

Para ver las redes disponibles:

```
nmcli dev wifi rescan && nmcli dev wifi list
```

Para ver las conexiones guardadas:

```
nmcli connection show
```

9. Conexión del LED RGB

El LED RGB utilizado es de ánodo común. En este tipo de LED el pin positivo va a 3.3V y cada canal se activa poniendo el pin GPIO en HIGH.

Canal	Pin GPIO (BCM)	Pin físico
Rojo	GPIO 17	Pin 11
Verde	GPIO 27	Pin 13
Azul	GPIO 22	Pin 15
Ánodo común	3.3V	Pin 1 o 17

10. Diagnóstico y Solución de Problemas

Problema	Diagnóstico y solución
El servicio no arranca	Revisar logs: <code>journalctl -u fireguard-setup -n 50 --no-pager</code>
Error: No such file or directory (best.onnx)	Verificar que best.onnx está en <code>/home/fisica1/sistema_incendios/</code>
Error: name 'generar_codigo' is not defined	El archivo app.py está incompleto. Reemplazarlo con la versión corregida.
Bot no responde comandos de Telegram	Verificar conexión a internet: <code>ping -c 4 api.telegram.org</code>
Cámara no detectada	Verificar con: <code>ls /dev/video*</code> . Si no aparece, reconectar el USB.
LED no enciende	Verificar conexiones físicas y que aegnis-led.service esté corriendo
Stream web no carga	Verificar que <code>/tmp/aegnis_frame.jpg</code> existe y que fireguard-detect está corriendo
Error de DNS al conectar Telegram	La Pi no tiene internet. Verificar WiFi con: <code>nmcli connection show --active</code>

12. Resultados del Modelo

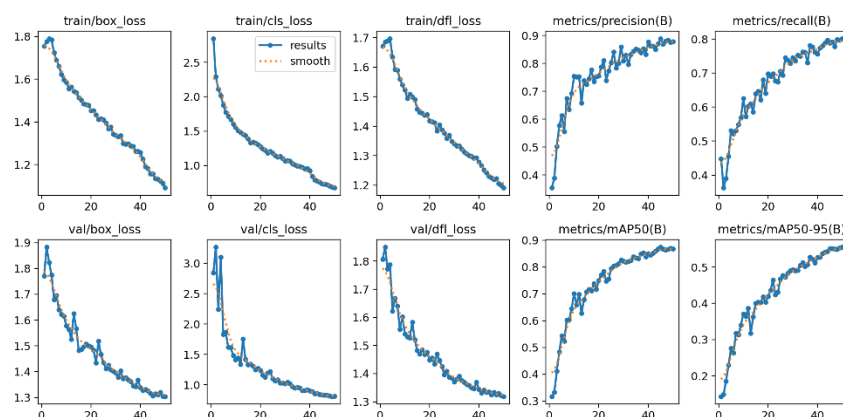
12.1 Métricas finales en el conjunto de prueba

Las siguientes métricas fueron obtenidas sobre el conjunto de prueba de 750 imágenes, el cual no fue visto por el modelo durante el entrenamiento ni la validación, por lo que representa una evaluación objetiva del desempeño real del sistema.

Métrica	Valor
mAP50	0.873
mAP50-95	0.548
Precision	0.883
Recall	0.788
AP clase fuego (0)	0.874
AP clase humo (1)	0.862

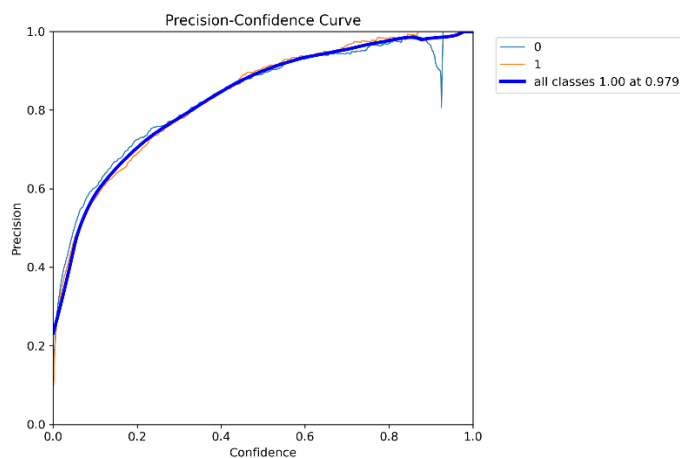
El mAP50 de 0.873 indica que el modelo detecta correctamente el 87.3% de los objetos presentes con un umbral de IoU del 50%. La precisión de 0.883 significa que el 88.3% de las detecciones realizadas son verdaderos positivos, manteniendo una tasa de falsas alarmas baja. El recall de 0.788 indica que el sistema identifica el 78.8% de todos los eventos reales de fuego o humo presentes en las imágenes de prueba.

12.2 Curvas de entrenamiento



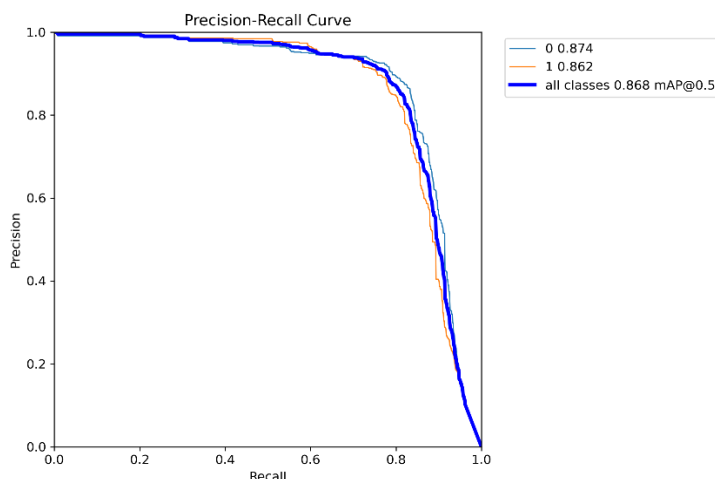
Las curvas de entrenamiento muestran la evolución de las funciones de pérdida y las métricas de desempeño a lo largo de las 50 épocas. Las pérdidas de localización (box_loss), clasificación (cls_loss) y distribución focal (dfl_loss) presentan una reducción progresiva y estable tanto en entrenamiento como en validación, sin evidencia de sobreajuste ya que ambas curvas mantienen trayectorias paralelas. Las métricas de precisión, recall, mAP50 y mAP50-95 muestran una tendencia ascendente sostenida a lo largo de todo el entrenamiento, alcanzando sus valores máximos en la época 50 sin plateau pronunciado, lo que indica que el modelo podría continuar mejorando con épocas adicionales.

12.3 Curva Precisión-Confianza



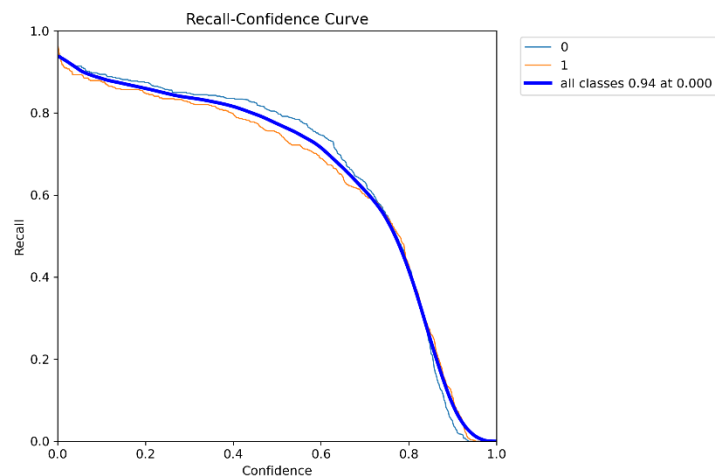
La curva Precisión-Confianza muestra cómo varía la precisión del modelo en función del umbral de confianza utilizado para aceptar detecciones. La curva global alcanza precisión 1.00 a un umbral de 0.979, lo que significa que con un umbral suficientemente alto el modelo puede lograr cero falsas alarmas. Ambas clases presentan comportamiento muy similar en toda la curva. Con el umbral operativo configurado en 0.5, la precisión se sitúa entre 0.85 y 0.93, rango óptimo para la aplicación.

12.4 Curva Precisión-Recall (Área bajo la curva)



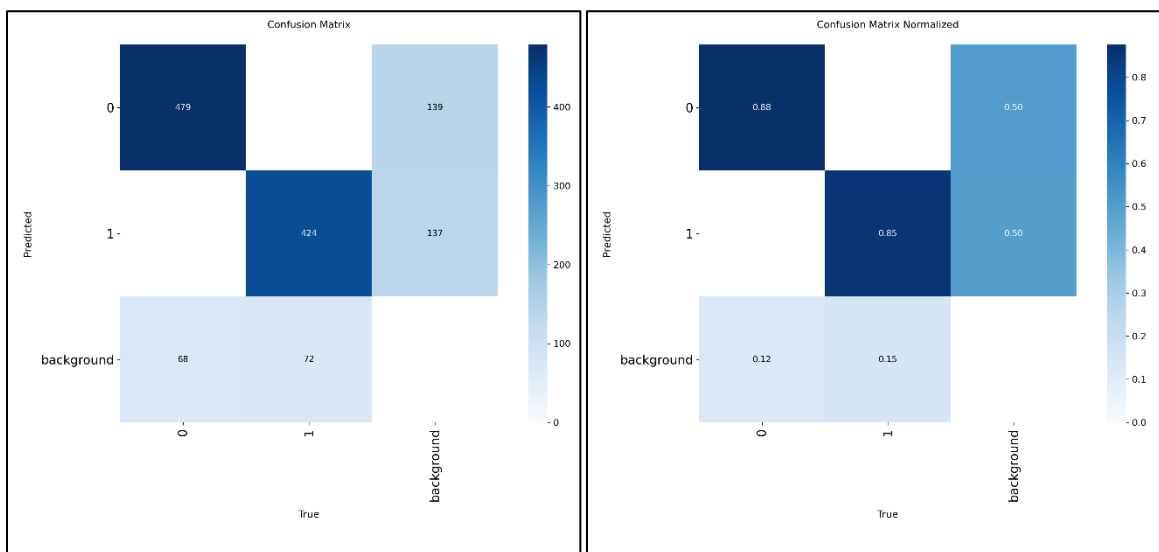
La curva Precisión-Recall refleja el equilibrio entre ambas métricas al variar el umbral de detección. El área bajo esta curva corresponde al Average Precision (AP) de cada clase: 0.874 para fuego y 0.862 para humo, con un promedio global de 0.868 mAP@0.5. La curva mantiene precisión cercana a 1.0 para valores de recall de hasta 0.70, comenzando a caer únicamente cuando se intenta recuperar los casos más difíciles o ambiguos del dataset. Esta forma indica que el modelo es altamente confiable en la mayoría de los escenarios de detección.

12.5 Curva Recall-Confianza



La curva Recall-Confianza muestra cómo decrece el recall conforme se incrementa el umbral de confianza requerido. A umbral 0.0 el recall global alcanza 0.94, indicando que el modelo es capaz de recuperar el 94% de los eventos reales si se acepta cualquier detección. La clase fuego mantiene recall ligeramente superior a la clase humo en todos los rangos, lo que sugiere que las características visuales del fuego son más distintivas para el modelo.

12.6 Matriz de confusión



La matriz de confusión revela el comportamiento detallado por clase sobre el conjunto de prueba. En valores absolutos, la clase fuego registra 479 verdaderos positivos y 68 casos no detectados (falsos negativos), representando una tasa de omisión del 12.4%. La clase humo presenta 424 verdaderos positivos con 72 casos no detectados, equivalente a una tasa de omisión del 14.5%. No se observan confusiones entre ambas clases entre sí, lo que confirma que el modelo distingue correctamente el fuego del humo cuando realiza una detección.

En la versión normalizada, los valores diagonales de 0.88 para fuego y 0.85 para humo confirman el desempeño sólido del modelo. Los valores de 0.50 en la columna background indican que cuando el modelo detecta algo en una región de fondo, la asignación entre clases es equiprobable, siendo este el principal vector de mejora para versiones futuras. En el sistema operativo este comportamiento se mitiga mediante el cooldown de 30 segundos entre alertas y la presentación de la fotografía al usuario para validación visual.

12.7 Ejemplos de detección



Los ejemplos de detección muestran el comportamiento del modelo en una amplia variedad de condiciones representativas de entornos interiores reales: diferentes ángulos de cámara típicos de sistemas de vigilancia, condiciones de iluminación variadas incluyendo escenas nocturnas y zonas oscuras, diferentes tamaños de foco de fuego desde llamas pequeñas hasta incendios más desarrollados, y entornos heterogéneos como laboratorios, aulas, oficinas y áreas industriales. Los niveles de confianza reportados oscilan mayoritariamente entre 0.7 y 0.9, consistente con las métricas globales obtenidas.

Ejemplos de detección y mensajería por Telegram:



 PELIGRO ALTO

• Incendio — 83.4% confianza

03:21 PM



 PELIGRO ALTO

• Incendio — 73.6% confianza

08:53 AM



 PELIGRO ALTO

• Incendio — 81.0% confianza

12:44 PM

13. Referencias

- Putra, A. K. (2025). Indoor Fire Smoke Dataset. Zenodo.
<https://doi.org/10.5281/zenodo.15826133>
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. GitHub.
<https://github.com/ultralytics/ultralytics>
- ONNX Runtime. (2024). ONNX Runtime documentation. <https://onnxruntime.ai>
- Raspberry Pi Foundation. (2024). GPIO documentation.
<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>
- Flask. (2024). Flask documentation. <https://flask.palletsprojects.com>
- NetworkManager. (2024). nmcli man page.
<https://networkmanager.dev/docs/api/latest/nmcli.html>